TheBoys

Gerenciador de Tarefas Acadêmicas

Vue 3 + TypeScript + Vite

Carlos Eduardo · Igor Evangelista Cruz · Heitor Mota da Cruz · João Vitor Fernandes

Site
`theboys-kl6z.onrender.com`

API
`theboys-api-d12x.onrender.com`

O que o app faz

- Lista e **quadro Kanban** das tarefas · página de cada projeto
- Cadastro, edição e **exclusão com confirmação**
- **Busca** sem maiúsculas nem acentos e **filtros** compartilhados entre lista e quadro
- Status pelo **seletor no cartão** ou **arrastando** (desafio opcional)
- Estados de **carregando**, **vazio**, **erro** e **não encontrado** em toda tela que busca dados
- Dois temas: **Herói** (claro) e **Vilão** (escuro)
- **Publicado:** site e API no Render

Em números

6 · 46	marcos · issues no GitHub Projects
11 · 14	telas (src/views) · componentes (src/components)
11	funções tipadas em 1 módulo de API (src/api.ts)
93 → 100	Lighthouse acessibilidade nas telas auditadas (Marco 5)

`src/tipos.ts · linhas 1 e 12-23`

```ts
export type Status = 'a-fazer' | 'em-andamento' | 'em-revisao' | 'concluida'

export interface Tarefa {
  id: string
  projetoId: string
  titulo: string
  descricao: string
  status: Status
  prioridade: Prioridade
  responsavel: string
  prazo: string
}

export type NovaTarefa = Omit<Tarefa, 'id'>
```

Quem cria não inventa id: quem gera é a API.

Vue: o modelo conceitual

Componente de arquivo único

.vue = script + template + estilo

`CartaoTarefa.vue · FormularioTarefa.vue`

Reatividade

mudou o dado, a tela se redesenha sozinha

`reactive() dos filtros · ref() de “salvando”, “erro”`

Props descem, eventos sobem

o cartão recebe tarefa e emite os avisos

`:tarefa · atualizada · excluida`

Slot com escopo

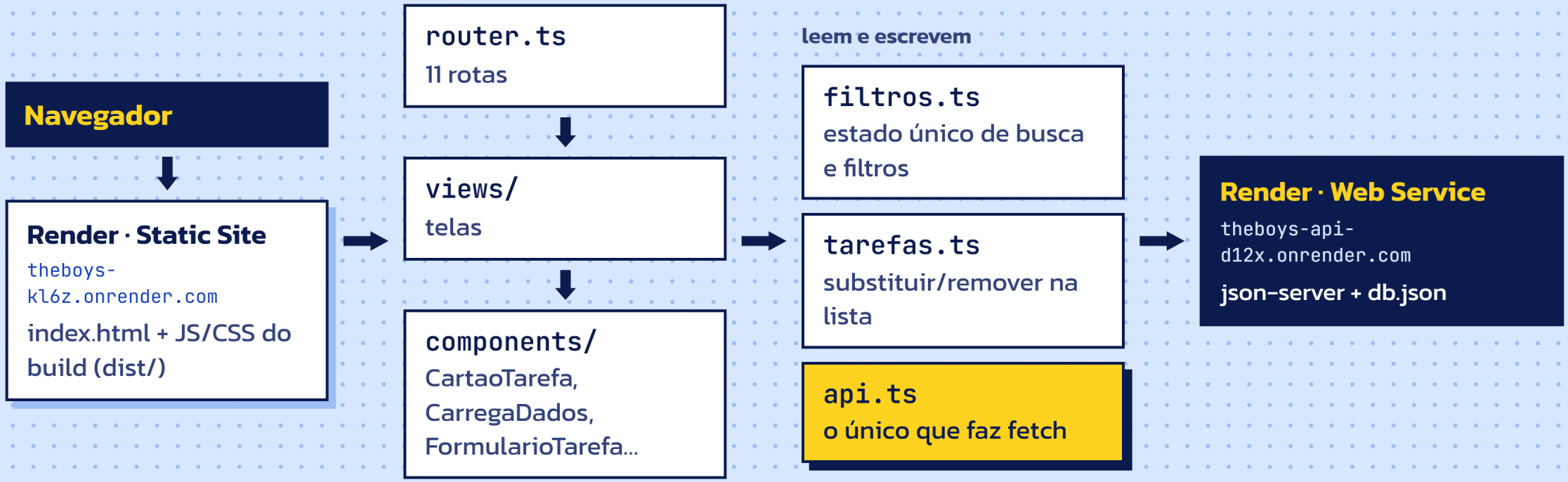
o pai decide como mostrar o dado que o filho buscou

`CarregaDados busca, a tela desenha`

`src/main.ts · linha 7`

```
createApp(App).use(router).mount('#app')
```

Arquitetura final



Três camadas: telas, componentes reaproveitados e módulos TypeScript sem tela. **Só o api.ts conhece o endereço da API.**

Como chegamos aqui

Marco 1	Marco 2	Marco 3	Marco 4	Marco 5	Marco 6	Extra
Repositório, Vite, json-server, Projects Contrato de dados fixo desde o primeiro dia	**Listagem e rotas** Tarefas de projeto via _embed (o ?projetoId=1 volta vazio)	**Cadastro e edição** "Um formulário, duas telas"	**Busca, filtros e quadro** "Onde o estado mora?" · "Derivado não é estado"	**Exclusão, erros e estados** "Quando algo dá errado" · Lighthouse 93 → 100	**Publicação** API e site no Render, rotas abrindo direto	**Arrastar no quadro + tema Herói/Vilão** Desafio só depois do escopo obrigatório

Cada marco reaproveitou o anterior em vez de reescrever.

Decisão 1: uma porta só para a API, e ela evoluiu

Antes · Marco 2 · src/api.ts · commit 86fde7d · linhas 7-12

```
async function buscar<T>(caminho: string): Promise<T> {
  const resposta = await fetch(`${BASE_URL}${caminho}`)
  if (resposta.status === 404) throw new ErroNaoEncontrado(caminho)
  if (!resposta.ok) throw new Error(`Falha na API (${resposta.status})`)
  return resposta.json() as Promise<T>
}
```

Depois · Marco 5 · src/api.ts · linhas 28-39

```
// O fetch só rejeita quando não há resposta nenhuma; erros HTTP chegam como resposta.
async function requisitar(caminho: string, opcoes?: RequestInit): Promise<Response> {
  let resposta: Response
  try {
    resposta = await fetch(`${BASE_URL}${caminho}`, opcoes)
  } catch {
    throw new ErroSemConexao()
  }
  if (resposta.status === 404) throw new ErroNaoEncontrado(caminho)
  if (!resposta.ok) throw new ErroApi(resposta.status)
  return resposta
}
```

Decisão 2: um formulário, duas telas

src/components/FormularioTarefa.vue · linhas 9-11

```ts
// Sem `tarefa` o formulário cadastra; com `tarefa` ele edita.
const props = defineProps<{ tarefa?: Tarefa; enviando?: boolean; projetoIdInicial?: string }>()
const emit = defineEmits<{ salvar: [dados: NovaTarefa]; cancelar: [] }>()
```

src/components/FormularioTarefa.vue · linhas 18-20

```ts
const form = reactive<NovaTarefa>({
  projetoId: props.tarefa?.projetoId ?? props.projetoIdInicial ?? '',
  titulo: props.tarefa?.titulo ?? '',
```

O formulário
valida e emite salvar; não fala com a API

A tela decide
NovaTarefa → criarTarefa · EditarTarefa → atualizarTarefa

projetoIdInicial
aberto pelo projeto, ele já vem escolhido

Decisão 3: onde o estado mora

src/filtros.ts · linhas 7-11

```ts
export const filtros = reactive({
  busca: '',
  status: '' as Status | '',
  prioridade: '' as Prioridade | '',
})
```

Fora das rotas

Lista e quadro são rotas irmãs: a busca sobrevive à troca de visão.

src/filtros.ts · linhas 31-39

```ts
export function filtrarTarefas(tarefas: Tarefa[]): Tarefa[] {
  const termo = termoBusca()
  return tarefas.filter(
    (tarefa) ⇒
      (!termo || normalizar(tarefa.titulo).includes(termo)) &&
      (!filtros.status || tarefa.status === filtros.status) &&
      (!filtros.prioridade || tarefa.prioridade === filtros.prioridade),
  )
}
```

Derivado não é estado: a lista filtrada é calculada, nunca guardada.

Decisão 4: o dado desce, o aviso sobe

src/components/SeletorStatus.vue · linhas 23-24

```
    const salva = await atualizarStatusTarefa(props.tarefa.id, novo)
    emit('atualizada', salva)
```

src/views/ListaTarefas.vue · linhas 36-42

```
      <CartaoTarefa
        v-for="tarefa in filtrarTarefas(dados)"
        :key="tarefa.id"
        :tarefa="tarefa"
        @atualizada="substituirTarefa(dados, $event)"
        @excluida="removerTarefa(dados, $event)"
      />
```

> "O cartão não muda de coluna; muda o status, e a coluna recalcula."

Lição do Marco 5

Evento sem ouvinte não dá erro no Vue. Hoje toda tela com cartões ouve os dois eventos.

Decisão 5: toda tela tem quatro estados

Carregando	Erro	Não encontrado	Sucesso

src/components/CarregaDados.vue · linhas 42-47

```vue
<template>
  <IndicadorCarregamento v-if="carregando" :mensagem="mensagem" :formato="formato" />
  <AvisoNaoEncontrado v-else-if="erro instanceof ErroNaoEncontrado" />
  <AvisoErroApi v-else-if="erro" :erro="erro" :tentativas="tentativas" @tentar-de-novo="tentarDeNovo" />
  <slot v-else-if="dados" :dados="dados as T" />
</template>
```

generic="T": o dado chega pelo slot já tipado; "Tentar de novo" busca sem recarregar.

No Render: a API gratuita dorme, e o primeiro acesso cai no "carregando", não numa tela quebrada.

Decisão 6: atualização pessimista, até no arraste

src/components/CartaoTarefa.vue · linhas 26-33

```
  try {
    await excluirTarefa(id)
    emit('excluida', id)
  } catch {
    erro.value = 'Não foi possível excluir a tarefa. Tente novamente.'
  } finally {
    excluindo.value = false
  }
```

src/arraste.ts · linhas 222-229

```
    // segura o cartão sobre a coluna enquanto a API responde
    estado.fase = 'gravando'
    const gravou = await mover(tarefaId, statusAlvo)
    if (!atual()) return
    if (!gravou) {
      await voltar()
      return
    }
```

A tela espera a API
nunca mostra algo que não aconteceu

No arraste também
o cartão fica segurado sobre a coluna; se a API falhar, volta

Pointer Events
e não o drag-and-drop nativo, para desenhar herói e vilão

Acessibilidade
o seletor de status faz o mesmo por teclado e leitor de tela

src/views/QuadroTarefas.vue · linhas 33-34

```
// Os desenhos do arraste são pesados: só são baixados quando o quadro abre.
const PalcoArraste = defineAsyncComponent(() => import('../components/arraste/PalcoArraste.vue'))
```

Publicação no Render

1 O endereço saiu do código

issue 39

```
// src/api.ts · linha 3
const BASE_URL = import.meta.env.VITE_API_URL ?? 'http://localhost:3000'
```

```
# Endereço da API em produção (json-server hospedado no Render)
VITE_API_URL=https://theboys-api-d12x.onrender.com
```

2 A API virou Web Service

issue 40 · mesmo json-server e db.json

```
// package.json · linha 10
    "api": "json-server db.json --host 0.0.0.0"
```

3 O site virou Static Site

issues 42 e 43 · rotas abrindo direto

```
/* → /index.html · Rewrite
```

Sem a regra: **404** · com a regra: **200**

Por que não Vercel: não mantém servidor ligado nem grava em disco

Plano gratuito: a API dorme após ~15 min; 1º acesso leva ~50 s

Dados: voltam ao db.json quando o serviço reinicia

Fácil, difícil e o que faríamos diferente

Foi fácil

Começar: npm create vite@latest com vue-ts já trouxe TypeScript, servidor e build

Reatividade: v-model + reactive() resolveram busca e filtros sem biblioteca extra

Reaproveitar: um cartão, um formulário e um CarregaDados para todas as telas

Foi difícil

O fetch **não falha** em 404 e 500, só quando não há resposta

Erros silenciosos do Vue: evento emitido sem ninguém ouvindo

O json-server lê ?projetoId=1 como número; o contrato guarda "1": resolvemos com _embed

O arraste: Pointer Events, colunas com rolagem, Esc e voltar se a API falhar

Publicar: json-server fora da Vercel e rotas dando 404

Integrar branches: vários merges da main em cada issue

Faríamos diferente

Busca e filtros **na URL**: sobreviveriam ao F5 e virariam link

Testes automatizados desde o início (não temos nenhum)

Banco persistente em produção, em vez de dados que voltam ao original

Lighthouse desde o Marco 2, não só no Marco 5

Ao vivo

theboys-kl6z.onrender.com

Obrigado!

Site · theboys-kl6z.onrender.com

API · theboys-api-d12x.onrender.com

Código · github.com/carlosarantes123/TheBoys

Perguntas?

Aponte a câmera